DATA STRUCTURE AND ALGORITHMS

* Data structure and Algorithms is one of the most fundamental subjects in computer science and software engineering.

* It focuses on how data is organised, stored and processed and manipulated efficiently.

* DSA provides programmers with the knowledge and techniques required to solve computational problems in the most efficient manner while minimizing the use of memory and processing time.

What is Data?

Data refers to raw facts, figures, symbols or values that can be processed to produce meaningful information.

Ex: Student marks, Employee salaries, etc.

What is Data Structure?

A data structure is a logical or mathematical model use to organise data in memory in such a way that operations such as insertion, deletion, searching, sorting and updating can be performed efficiently.

Ex: Library, ATM software, Student database, etc.

What is an Algorithm?

An algorithm is a finite sequence of well-defined instructions use to solve a problem or perform a particular task.

Ex: Search, sort, insert, update, delete

Characteristics of Good Algorithm

(i) Input
(ii) Output
(iii) Definiteness
(iv) Finiteness
(v) Effectiveness.

Relationship b/w Data structures and Algorithms

* Data structure determines how data is stored.

* Algorithms determines how data is processed.

Data Terminologies Definition

Information: Processed data that is meaningful and useful for decision making.

Data Item: A single unit of data representing one value or fact.

Group Item: A collection of related elementary data items treated as one unit.

Elementary item :- the smallest invisible unit of data that can't be further subdivided.

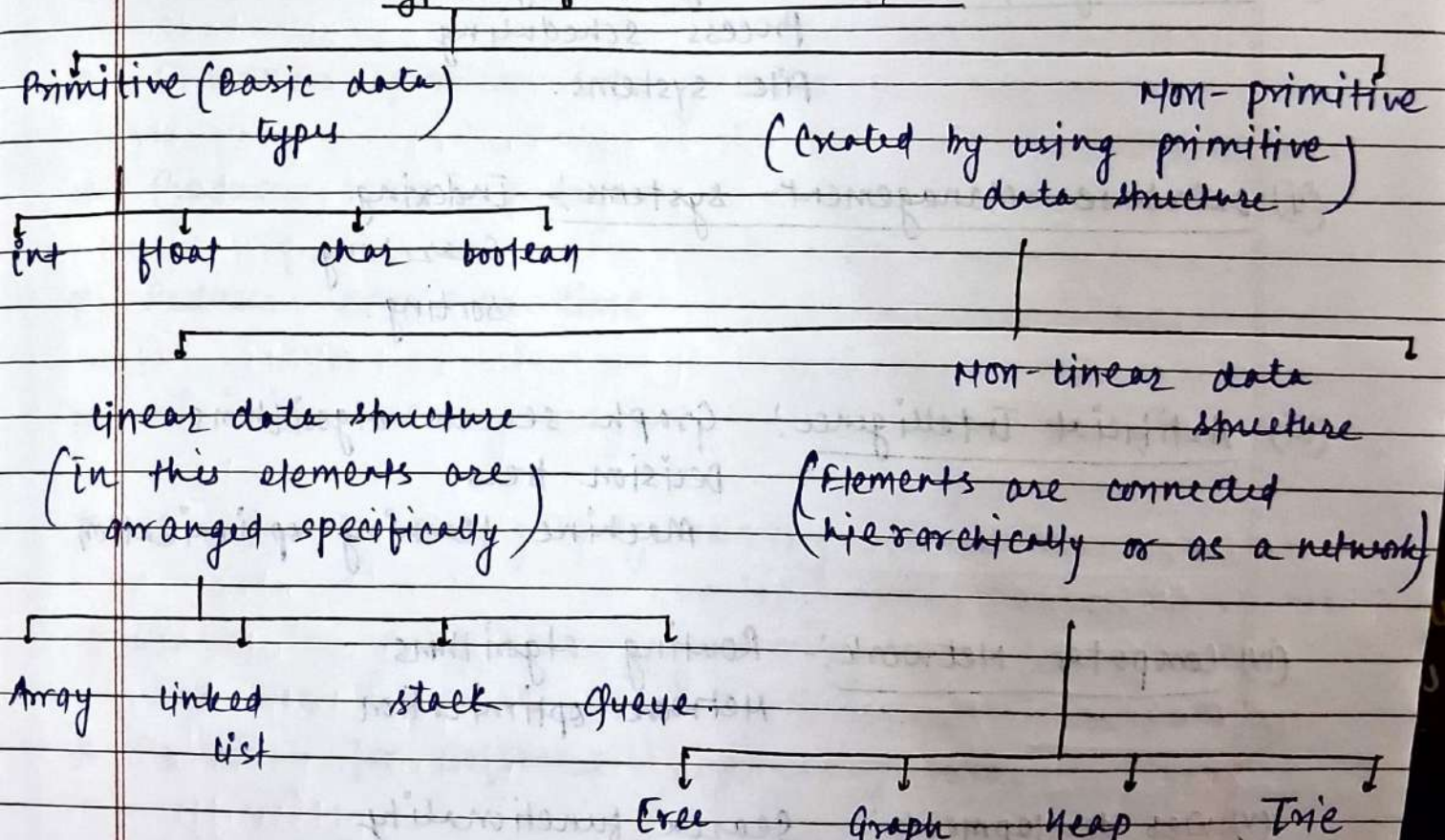
Entity :- A real world object, person, place or concept about which data is stored.

Field :- A specific attribute or characteristic of an entity that stores one piece of data.

Record :- A collection of related fields describing a single entity.

File :- A collection of related records stored together as a single unit.

Types of Data structures



Importance of DSA

DSA is important because it helps developers write programs that are:

* Faster
* More efficient
* Easier to maintain
* less memory consuming
* More scalable.

Applications of DSA

(i) Operating systems :- Memory Management
Process scheduling
File systems.

(ii) Database Management systems :- Indexing
Searching
Sorting.

(iii) Artificial Intelligence :- Graph search algorithms
Decision trees
Machine learning, optimization

(iv) Computer Network :- Routing algorithms
Network optimization

(v) Web development :- Search functionality
Recommendation systems
Session management

(vi) Mobile Applications :- Contact management
Maps & Navigation
Chat application.

(vii) Game development :- Pathfinding
Collision detection
AI movement.

(viii) Cybersecurity :- Encryption Algorithms
Hashing
Digital signatures.

Advantages of DSA

* Improves logical thinking
* Enhances problem solving skills
* Produces optimized code
* Reduces execution time
* Minimizes memory usage
* Builds a strong programming foundation
* Useful in competitive programming
* Essential for software engineering roles.

Abstract Data Type (ADT)

Ajanta
Page No.
Date

It is a logical or mathematical model that defines a data type by specifying what operations can be performed on the data and what those operations do without specifying how the data is implemented.

Components of ADT :-

(i) **Data** :- The information stored in the data structure.

(ii) **Operations** :- The functions that can be performed on the data.

Features of ADT :-

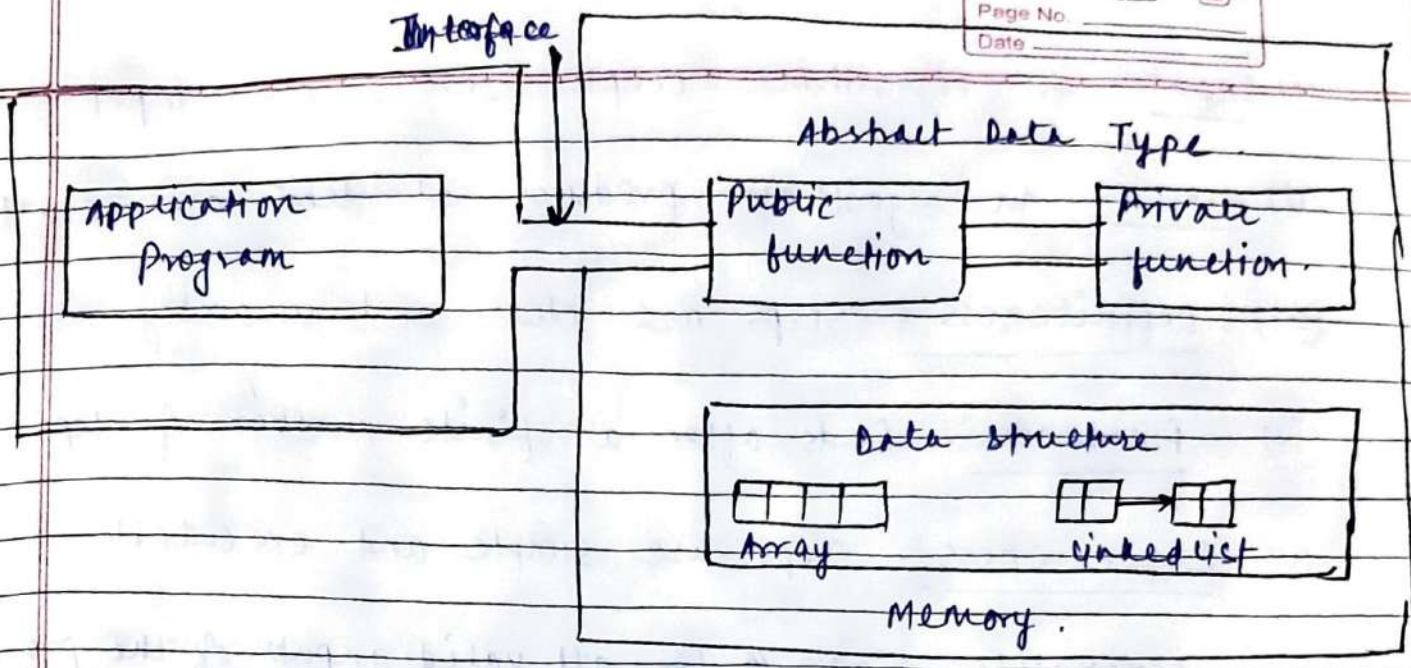
(i) **Abstraction** :- Only essential information is exposed.

(ii) **Encapsulation** :- Data & operations are grouped together.

(iii) **Data Hiding** :- Internal representation is hidden from the user.

(iv) **Implementation Independence** :- The same ADT can have different implementations.

(v) **Modularity** :- Components can be developed and tested independently.



ADT and Data structure Relationship

Abstract Data Type (ADT) → Defines operation → Data structure → Implements operations.

ADT V/S Data structure

Abstract Data Type (ADT)	Data structure
* logical concept	* Physical Implementation
* Defines operations	* Implements operations
* focuses on what	* focuses on how
* Independent on implementation	* Depends on implementation
* Ex: Stack ADT, Queue, List	* Ex: Stack using array or linked list, Tree.

Characteristics of algorithms

(i) **Input**: An algorithm accepts zero or more input.

(ii) **Output**: An algorithm produces at least one result.

(iii) **Definiteness**: steps are clear and unambiguous.

(iv) **Finiteness**: Ends after a finite number of steps.

(v) **Effectiveness**: steps are simple and executable.

(vi) **Generality**: Works for all valid inputs of the problem.

Basic operations on data structure

(i) **Transversal**: It means visiting each element of a data structure exactly once to perform an operation such as displaying or processing data.

Ex: Print of elements of an array.

```
#include <stdio.h>
int main(){
    int arr[]={10,20,30,40,50};
    int n=5;

    printf("Array elements are:\n");
    for(int i=0; i<n; i++){
        printf("%d", arr[i]);
    }
    return 0;
}
```

(ii) **Insertion**:- It is the process of adding a new element to a data structure at a specified position.

Ex:- Insert an element at a given position.

```
#include <stdio.h>
int main(){
  int arr[]={10, 20, 30, 40};
  int n=4;
  int element=50;
  int position=4;
  for(int i=n ; i>=position ; i--){
    arr[i]=arr[i-1];
  }

  arr[position-1]=element;
  n++;

  printf("Array after insertion:\n");
  for(i=0 ; i<n ; i++){
    printf("%d", arr[i]);
  }
  return 0;
}
```

(iii) **Deletion**:- Deletion is a process of removing an element from a data structure.

Ex:- Delete an element from a given position.

```
#include <stdio.h>
int main(){

  int arr[10]={10, 20, 30, 40, 50};
  int n=5;
  int position=3;
```

```
for (int i= position -1 ; i<n-1 ; i++){
    arr[i]= arr[i+1]
    }
    n--;

    printf(" Array after deletion: \n");
    for (int i=0 ; i<n ; i++){
        printf("%d", arr[i]);
    }
    return 0;
}
```

(iv) Searching: It is a process of finding whether a particular element exists in a data structure and determining its location.

Ex:- Linear search

```
#include <stdio.h>
int main(){
int arr[]={10, 20, 30, 40, 50};
    int n=5;
    int key=40;
    int found=0;

for (int i=0; i<n ; i++){
    if (arr[i]==key){

        printf(" Element found at position %d", i+1);
        found=1;
        break;

    }
}
```

```
if (! found)
    printf ("Element not found");
    return 0;
}
```

(v) Sorting :- It is the process of arranging elements in ascending or descending order.

Ex:- Bubble sort (Ascending order).

```
#include <stdio.h>
int main(){
    int arr[]= {40, 10, 50, 20, 30};
    int n= 5;

    for (int i=0 ; i<n-1 ; i++){
        for (int j=0 ; j<n-i-1 ; j++){

            if (arr[j] > arr[j+1]){
                int temp = arr[j];

                arr[j]= arr[j+1];
                arr[j+1]= temp;
            }
        }
    }
    printf (" Sorted array: \n");
        for (int i=0; i<n ; i++){
            printf ("%d", arr[i]);
    }
    return 0;
    }
```

(vi) Merging:- It is the process of combining two data structures of the same type into a single data structure.

Ex:- Merge two arrays.

```
#include <stdio.h>
int main(){
    int arr1[]={10, 20, 30};
    int arr2[]={40, 50, 60};

    int arr3[6];

    int n1=3, n2=3;

    for(int i=0; i<n1; i++)
        arr3[i]=arr1[i];

    for(int i=0; i<n2; i++)
        arr3[n1+i]=arr2[i];

    printf("Merged array:\n");
    for(int i=0; i<n1+n2; i++)
        printf("%d", arr3[i]);

    return 0;
}
```

Algorithm analysis

It is the process of evaluating the performance and efficiency of an algorithm by studying the time & memory it requires to solve a problem.

Objective of Algorithm analysis:-

(i) To measure efficiency of an algorithm.

(ii) To compare different algorithm for the same problem.

(iii) To reduce execution time

(iv) To reduce memory usage

(v) To predict the performance for large input sizes.

Factors affecting the efficiency of algorithm

(i) Input Size:- Larger input increases execution time.

(ii) Time Complexity:- It determines how much time an algorithm takes to execute as the input size increases.

Ex:- Linear search → O(n)
Binary search → O(log n)

(iii) Space Complexity:- It refers to the amount of memory required by an algorithm during execution.

Ex:- An algorithm using only variable requires O(1) space

An algorithm using an extra array requires O(n) space.

(iv) Algorithm Design Technique :- The method used to design an algorithm affects its efficiency.

Ex:- Divide and Conquer → Merge sort ($O(n \log n)$)
Brute force → Bubble sort ($O(n^2)$)

(v) Data structure used :- The choice of data structure affects the speed of operations.

Ex:- Searching in a Hash Table → $O(1)$ avg. time.
Searching in a linked list → $O(n)$ time

Types of Algorithm Analysis

Time Complexity

* It measures the amount of time an algorithm takes to execute as the input size increases.
* Represented by Big O (O).

Ex:-
```
for (i=0; i<n; i++){
    printf("%d", i);
}
```

This loop runs n times

Time Complexity = $O(n)$.

Space Complexity

* It measures the amount of memory required by a algorithm during execution.

Ex:- int arr[n];

The algorithm uses to store n elements.

Space Complexity = $O(n)$.

Asymptotic Analysis

It studies the behaviours of an algorithm when the input size becomes very large.

Notation	Name	Meaning
O (Big O)	Upper Bound	Max^m time taken
Ω (omega)	lower Bound	Min^m time taken
Θ (Theta)	Tight Bound	Avg / Exact growth rate

Big O (O): A function is said to be O(g(n)) if there exists constant C & a constant n_0 such that:-

$$0 \leq f(n) \leq Cg(n) \quad \forall \quad n \geq n_0$$

f(n) → Actual running time
Cg(n) → Reference fxn
n_0 → Starting pt.

Ex: $f(n) = n^3 + 1 \quad g(n) = n^4$

$0 \leq n^3 + 1 \leq 1 \cdot n^4$ $\quad n_0 = 2$ (checking by putting values)

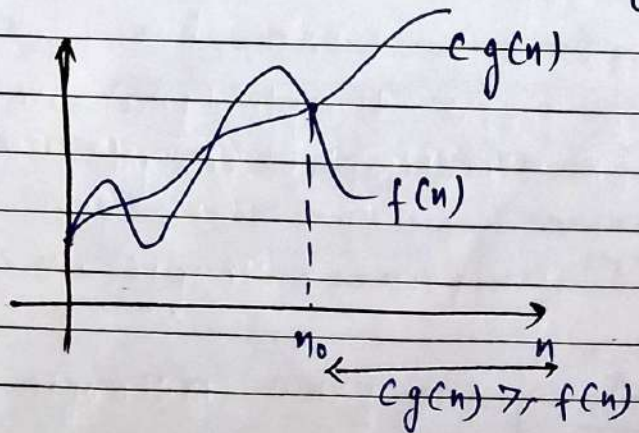
$C = 1$

Big O

$g(n) = n^3$ → Big O of $O(n^3)$

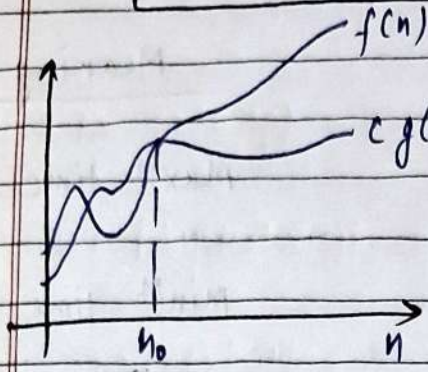
$0 \leq n^3 + 1 \leq 2n^3 \quad n_0 = 2$

$C = 2$



Big omega (Ω):- A function is said to be $\Omega(g(n))$ if there exists a constant C & constant n_0 such that

$$0 \leq C g(n) \leq f(n) \quad \forall\ n \geq n_0$$



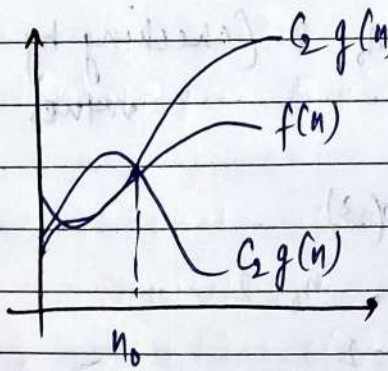
Ex:- $n^3 + 1 = \Omega(1)$ (Big omega of 1)

$$0 \leq 1 \leq n^3 + 1$$

Big Theta (Θ):- A function $f(n)$ is said to be $\Theta(g(n))$ if there exists constant C_1, C_2 & n_0 such that

$$\left.\begin{aligned} 0 &\leq f(n) \leq C_1 g(n) \\ 0 &\leq C_2 g(n) \leq f(n) \end{aligned}\right\} \quad n \geq n_0$$

$$C_2 g(n) \leq f(n) \leq C_1 g(n)$$



Best, Worst and Expected Case

Ajanta
Page No.
Date

sorted arr[n]

1	5	7	9	24

Algo 1
n comparisons.

(searching value of a in arr) a=8 → 0
a=9 → 1

on searching a=1

Time Complexity

* Best case → O(1) [$T_n = k$]
(That it find the no. at 1st place
∴ only one comparison
Does not depend on size of arr.)

* Worst case → O(n) [$T_n = nk$]
(worst case is that
it requires n comparisons to search the value of a)

* Avg case = $O\left(\frac{\sum \text{all possible runtimes}}{\text{total no. of possibilities}}\right)$

$T_n = O\left(\frac{k + 2k + 3k + 4k + \cdots + nk + nk}{(n+1)}\right)$

(k: found at 1st place; 2k: found at 2nd place; nk: found at nth place; nk: when element not found)

$T_n = \frac{k(1+2+\cdots+n) + nk}{(n+1)} = \frac{k\left[\frac{n(n+1)}{2} + n\right]}{(n+1)}$

$= k\left[\frac{(n^2+n)+2n}{2(n+1)}\right]$ (dominant term / non dominant term.)

$= k\left[\frac{n(n+1)}{2(n+1)}\right] = \frac{k}{2} n = O(n)$

(k/2: const.)

Time Complexity and Big O

Ajanta
Page No.
Date

let take a example of two algorithms to transfer data:-

Algo 1 By Internet ⟶ Time changes with size of data (input)

Algo 2 By physical visit ⟶ Time changes same w.r.t to size of data (input)

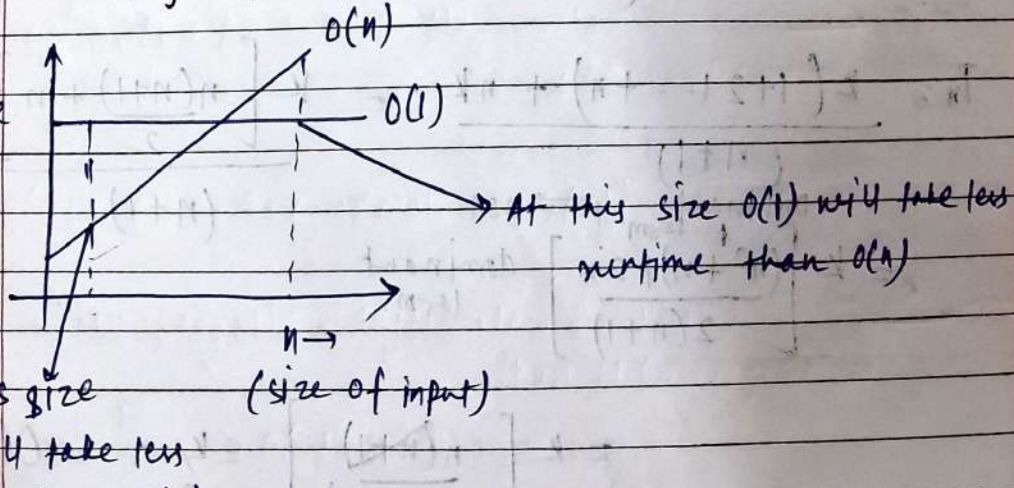
* $t(n)$ (n → input size) $= k_1 + k_2 + k_2 + k_3 = k_1 n^0 + k_2 n^0 + k_3 n^0 + k_4 n^0$ (most impactful term)
algo2 $= O(n^0) = O(1)$

* $t(n) = t_1 + t_2$ (speed of internet $= t_2$ kb/s)
algo1
$\Rightarrow t_1 + \left(\frac{n}{t_2}\right)$
$\Rightarrow t_1 n^0 + t_2 n^1$
↓ more impactful term.

n kb (data)
Speed = Dist./time
time = Dist./Speed = n/t_2

$t(n)$ algo 1 $= n^1 = O(n^1) = O(n) \rightarrow$ linear

Algo 1 ⟶ Internet ⟶ $O(n)$
Algo 2 ⟶ Physical visit ⟶ $O(1)$



At this size O(1) will take less runtime than O(n)

At this size O(n) will take less runtime than O(1)

Tricks to find Time Complexity.

(i) Drop the non-dominant terms.

(ii) Drop the constant terms.

(iii) Break the code into fragments.

Ques:- Find the time complexity of the funct function in the program.

Soln:-
```
#include <stdio.h>
void funct(int array[], int length){
                    int sum=0;
    sum+=array[i];  int product=1;        } f1 = k1
    for(int i=0; i<length; i++){
        sum+=array[i];                    } f2 = k2 n
    }
    for(int i=0; i<length; i++){
        product*=array[i];                } f3 = k3 n
    }
}
int main(){
    int arr[]={3,5,66};
    funct(arr,3);
    return 0;
}
```

$T_n = f_1 + f_2 + f_3$

Const

$\rightarrow k_1 + k_2 n + k_3 n$

$\rightarrow (k_2 + k_3) n \Rightarrow k_4 n \rightarrow O(n)$

Ques:- Find the time complexity of the func() function in the program.

Ajanta
Page No.
Date

Sol^n:-

```
void func(int n)
{
  int sum=0;
  int product=1;          ] → k1

  for(int i=0; i<n; i++){          ] → 0 to n
    for(int j=0; j<n; j++){        ] → 0 to n
      printf("%d, %d\n", i, j);    ] → k2
    }
  }
}
```

$T(n) = k_1 + nk_2(1+1+1+\cdots+1) + k_3$ (Const, Const)

$T_n = k_2 n^2 = O(n^2)$

Ques:- Which of the following is equivalent to O(N)

(a) $O(N+P)$ where $P < N/9$

(b) $O(9N-K)$

(c) $O(N+8\log N)$

(d) $O(N+M^2)$

Sol^n:- (i) $O(N+P) \longrightarrow P < N/9 \longrightarrow$ Non Dominant term $\longrightarrow O(N)$

$O(9N-K) \longrightarrow 9, K \rightarrow$ Const. $\longrightarrow O(N)$

$O(N+8\log N) \longrightarrow \log N \rightarrow$ less Dominant than N for large values $\longrightarrow O(N)$

$O(N+M^2) \longrightarrow O(N+M^2)$

N, M^2 → Both inputs

Abstract Data Type (ADT).

An abstract data type is a logical or mathematical model of data structure that specifies;

* what data is stored.
* what operations can be performed on the data.

Common ADTs

(i) List ADT → Insert, Delete, Search, Traverse

(ii) Stack ADT (LIFO) → Push, Pop, Peek

(iii) Queue ADT (FIFO) → Enqueue, Dequeue, Front

(iv) Deque ADT → Insert / Delete from both ends

(v) Set ADT → Add, Union, Remove, Intersection

(vi) Priority Queue ADT → Insert, Delete highest / lowest priority.

Arrays in DSA

* It is a linear data structure that stores a collection of elements of same data types in contiguous (adjacent) memory locations.
* Each element is accessed using its index.
* Size is usually fixed once the array is created (in static arrays)

Basic operations in arrays

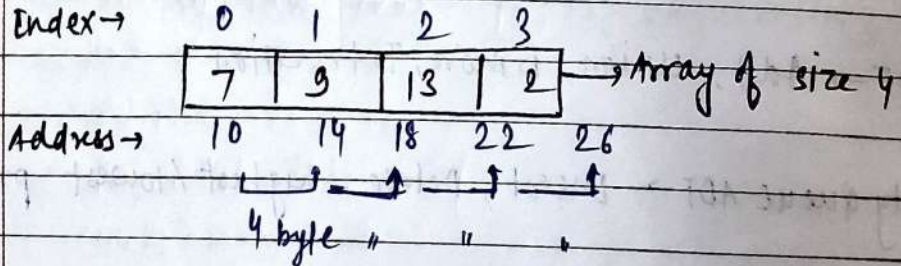
* Transversal :- Visit each element
* Insertion :- Add an element at specific position.
* Deletion :- Remove an element
* Searching :- find an element (Linear search, Binary search)
* Updating :- Change the value of an element.

minimal functionality

get(i) → get element i
set(i, num) → get element i to num.

Static arrays :- size cannot be changed

Dynamic arrays :- size can be changed



Structures in C

Ajanta
Page No.
Date

* Structures are user defined data types in C.
* Using structures allow us to combine data of different types together.
* It is used to create a complex data type which contains diverse information.
* They are very similar to arrays but structures can store data type of any type.

Defining a structure:

```
struct [structure_name] {
// data_type var1;
// data_type var2;
// data_type var3;
}
[structure_variables];
```

Declaring a structure

```
#include <stdio.h>
struct Employee {
 int id;
 char name[53];
 float marks;
};
struct Employee e1, e2;
 int main (){

 return 0;
 }
```

```
#include <stdio.h>
struct Employee {
 int id;
 char name[53];
 float marks;
}
 e1, e2;
int main(){
struct Employee tt;
 return 0;
 }
```

Accessing structure elements

* Array members are accessed using the subscript variable.

* Structure members are accessed using dot[.] operator (.) is called a structure member operator

Ex:

```
# include <stdio.h>   # include <string.h>
struct student {
int id;
int marks;
char fav_char;  char name[53];
};
int main(){
struct Student Harry, Ravi, Shubham,
Harry.id=1;
Harry.marks=66;
Harry.fav_char="p";
Ravi.id = 2
Ravi.marks = 76
Ravi.fav_char = "q";
Shubham.id = 3
Shubham.marks = 86
Shubham.fav_char = "r";
Strcpy(Harry.name," Harry Potter");
printf(" Harry got %d marks \n", Harry.marks);

printf("Harry.name");

return 0;
}
```

Ex:- Program to store details of 5 students using structures.

```
#include <stdio.h>
struct student {
    int roll;
    char name[50];
    int marks;
};
int main() {
    struct student s[5];

for (int i=0; i<5; i++) {
    printf("Enter details of student: %d\n", i+1);

    printf("Enter Roll no");
    scanf("%d", &s[i].roll);

    printf("Enter Name");
    scanf("%s", s[i].name);
    printf("Enter marks");
    scanf("%d", &s[i].marks);
}

printf("\n Student Details \n");
for (int i=0; i<5; i++) {
    printf("\n Student %d \n", i+1);
    printf("Roll No: %d \n", s[i].roll);
    printf("Name: %s \n", s[i].name);
    printf("Marks: %d \n", s[i].marks);
}
return 0;
}
```

Self Referential structures

* A referential structure is a structure that contain a pointer to another structure of the same data type.

* It is self referential bcoz the structure refers to itself through a pointer.

Ex:-
```
# include <stdio.h>
struct Node {
    int data;
    struct Node *next;
};

int main(){
 struct Node n1, n2;
 n1.data=10;
 n2.data=20;

 n1.next=&n2;
 n2.next=NULL;

 printf("First Node: %d\n", n1.data);
 printf("Second Node: %d\n", n1.next->data);

 return 0;
}
```

next is a pointer that stores the address of another struct Node.

Output: First Node: 10
Second Node: 20.

Pointers in C

A pointer is a variable that stores the memory address of another variable.

Uses of pointers:-

* Access memory directly.
* Modify variables through their addresses.
* Works with arrays & strings.
* Allocate memory dynamically.
* Create linked list, graphs & trees.

Ex:- Simple Pointer Program.

```
#include <stdio.h>
int main(){
  int x=10;
  int *p;

  p=&x;

  printf("value of x=%d\n",x);
  printf("Address of x=%p\n",(void *)&x);

  printf("value of p=%p\n",(void *)p);
  printf("value pointed by p=%d\n",*p);

  return 0;
}
```

Ex:- Changing a variable using a pointer.

Ajanta
Page No.
Date

```
#include <stdio.h>
int main(){
 int x=10;
 int *p;
 p=&x;
 *p=50;
 printf("x=%d\n",x);
 return 0;
 }
```

Output:- x=50

Ex:- Swap Two Numbers (Pointers are commonly used to modify values inside functions)

```
#include <stdio.h>
void swap(int *a, int *b){
   int temp;

   temp=*a;
   *a=*b;
   *b=temp;
   }
int main(){
   int x=10;
   int y=20;
printf("Before swap: x=%d ,y=%d\n",x,y);

 swap(&x,&y);
 printf("After swap: x=%d , y=%d\n",x,y);

 return 0;
}
```

Ex:- Pointers And Arrays (The name of array can be used as an address of its first element in many expressions)

```
#include <stdio.h>
int main(){
int arr[]={10, 20, 30, 40, 50};

int *p=arr;
for(int i=0; i<5; i++){
    printf("%d ", *(p+1));
}
return 0;
}
```

p → arr[0]
*p → 10
*(p+1) → 20
*(p+2) → 30
*(p+3) → 40
*(p+4) → 50

Ex:- Pointers And Structures

```
#include <stdio.h>
struct Student {
    int roll;
    float marks;
};
int main(){
struct Student s={101, 85.5};
struct Student *p=&s;

printf("Roll No= %d\n", p->roll);
printf("Marks= %.2f\n", p->marks);

return 0;
}
```

Dynamic Memory Allocation in C

* Dynamic Memory Allocation is a process of allocating memory during program execution (runtime).

* In C, dynamic memory is allocated from the heap using functions provided by the <stdlib.h> header file.

main functions:-

* malloc():- Memory allocation
* calloc():- Contiguous allocation
* realloc():- Reallocation
* free():- Deallocation.

* Dynamic memory allocation is a way in which the size of data structure can be changed during the runtime.

* **malloc():-**

It allocates a single large block of memory of the specified size. It returns a pointer of type void which can be cast into a pointer of any form.

```
int *ptr = (int *) malloc (100 * sizeof (int));
```

* **calloc():-**

It allocates multiple block of memory, each of same size, and sets all allocated memory to zero.

```
int *ptr = (int *) calloc (100, sizeof (int));
```

* realloc():
It is used to dynamically change the memory allocation of a previously allocated memory.

ptr = realloc (ptr, 200 * sizeof(int));

* free():
Dynamically allocated memory can't be freed automatically, we must explicitly use free() to release the memory back to the system to prevent memory leaks.

free (ptr);

Ex: malloc():

```
#include <stdio.h>
#include <stdlib.h>
int main() {
  int n;
  int *p;
printf("Enter no. of elements:");
 scanf("%d", &n);

p=malloc(n* sizeof(int)); # Allocates enough memory for 5 integers.

if (p==NULL) {
  printf("Memory allocation failed\n");
    return 1;
  }
printf("Enter element\n");
 for(int i=0; i<n; i++) {
     scan("%d", &p[i]);
   }
printf("The elements are:\n");
for(int i=0; i<n; i++) {
  printf("%d", p[i]);
}
 free(p);
return 0;
}
```

Ex: calloc()

```
#include <stdio.h>
#include <stdlib.h>

int main(){
  int *p;
  p= calloc(5, sizeof(int)); # Allocates memory for 5 integers and initializes them to zero.
  if (p==NULL){
    printf("Memory allocation failed\n");
    return 1;
  }
  for(int i=0; i<5; i++){
    printf("%d", p[i]);
  }

  free(p);
  return 0;
}
```

Ex: realloc()

```
#include <stdio.h>
#include <stdlib.h>

int main(){
  int *p;

  p= malloc(3* sizeof(int));
  if (p==NULL){
    return 1;
  }
  p[0]=10;
  p[1]=20;
  p[2]=30;
  p=realloc(p, 5* sizeof(int));
  if (p==NULL)
    return 1;
  p[3]=40;
  p[4]=50;
  for(int i=0; i<5; i++){
    printf("%d", p[i]);
  }

  free(p);
  return 0;
}
```

Matrix Multiplication

It is an operation in which two matrices multiplied to produce a thaird matrix

$A(m \times n) * B(n \times p) = C(m \times p)$

No of columns of 1st matrix = No of rows of IInd matrix.

```
#include <stdio.h>
int main(){
  int a[3][3], b[3][3], c[3][3]={0};
int i, j, k;

printf("Enter first matrix:\n");
for(i=0; i<3; i++)
  for(j=0; j<3; j++)
    scanf("%d", &a[i][j]);
```

Entering 1st matrix

```
printf("Enter second matrix:\n");
for(i=0; i<3; i++)
  for(j=0; j<3; j++)
    scanf("%d", &b[i][j]);
```

Entering IInd matrix

```
for(i=0; i<3; i++)
 for(j=0; j<3; j++)
  for(k=0; k<3; k++)
   c[i][j]+= a[i][k] * b[k][j];
```

Matrix Multiplication

```
printf("Result\n");
for(i=0; i<3; i++){
 for(j=0; j<3; j++)
   printf("%d", c[i][j]);
   printf("\n");
}
 return 0;
}
```

Printing Result

Linear Search

Linear search is a simple searching algorithm in which each element of an array is checked sequentially from the beginning to the end until the required element is found or all elements have been checked.

Linear Search algorithm :-

Step1 :- Start

Step2 :- Set i=0

Step3 :- Compare a[i] with search element x

Step4 :- if a[i]==x : Then element is found return its pos"/index

Step5 :- Otherwise increment i. (i=i+1)

Step6 :- Repeat until the array ends.

Step7 :- If element not found return -1.

Step8 :- Stop.

Pseudo Code for Linear Search

```
(1) for i ← 0 to n-1
(2)     IF A[i] = key THEN
(3)         RETURN i
(4)     END IF
(5) END FOR
(6) RETURN -1
```

Linear Search complexity table

Case	Time Complexity	Space Complexity
Best Case	O(1)	O(1)
Worst Case	O(n)	O(1)
Average Case.	O(n)	O(1)

Advantages:- * Works on sorted / unsorted arrays
* Suitable for small datasets
* Can be used with arrays or other sequential data.
* Requires O(1) auxiliary space.

```
#include <stdio.h>
int main(){
int a[100], n, x, i;

printf("Enter no. of elements:");
scanf("%d", &n);

printf("Enter elements:");
for(i=0; i<n; i++)
scanf("%d", &a[i]);

printf("Enter element to search:");
scanf("%d", &x);

for(i=0; i<n; i++){
  if(a[i]==x){
    printf("Element found at index %d\n", i+1);
    return 0;
  }}
printf("Element not found");
  return 0;
}
```

Binary search

It is a searching algorithm used to find an element in an sorted array. It repeatedly divides the search range into two halves.

Binary search Algorithm

Step1: start

Step2: set low=0 and high= n-1

Step3: Repeat while low <= high:
Calculate the middle position:=
mid=(low+high)/2

Step 4: If A[mid]==key, then element is found.
If key < A[mid], search the left half by setting
high= mid-1

Otherwise search the right half by setting
low= mid+1

Step5: If low > high then Element not found

Step6: Stop.

Complexity of Binary search

Case:	Time Complexity
Best Case	O(1)
Worst Case	O(log n)
Average Case	O(log n)
Space (iterative)	O(1)

```
# include <stdio.h>
int main(){
   int a[100], n, x, low, high, mid, i;

printf(" Enter no. of elements:");
 scanf("%d", &n);

printf(" Enter sorted element:");
 for(i=0; i<n; i++)
   scanf("%d", &a[i]);

printf(" Enter element to search");
 scanf("%d", &x);

 low= 0;
 high= n-1;

while (low<high){
   mid=(low+high)/2;

if (a[mid]==x){
 printf("Element found at position %d\n", mid+1);
   return 0;
   }
else if (a[mid]>x)
   high= mid-1;
else
   low= mid+1;
}
 printf(" Element not found");
 return 0;
}
```

Difference between Linear Search & Binary Search

Linear Search	Binary Search
* Checks each element one by one until the original target is found.	* Repeatedly divides the search range in half to find the target.
* Works on both sorted and unsorted array.	* Requires the data to be sorted.
* Sequential search from the beginning to end.	* Compares the middle element & eliminates half of the remaining elements each step.
* Simple and easy to implement.	* Slightly more complex than linear search.
* Time Complexity :- Best Case → O(1) Worst Case → O(n) Avg. Case → O(n)	* Time Complexity :- Best Case → O(1) Worst Case → O(log n) Avg. Case → O(log n)

Sorting of an Array.

* It is the process of arranging the elements of an array in a particular order.

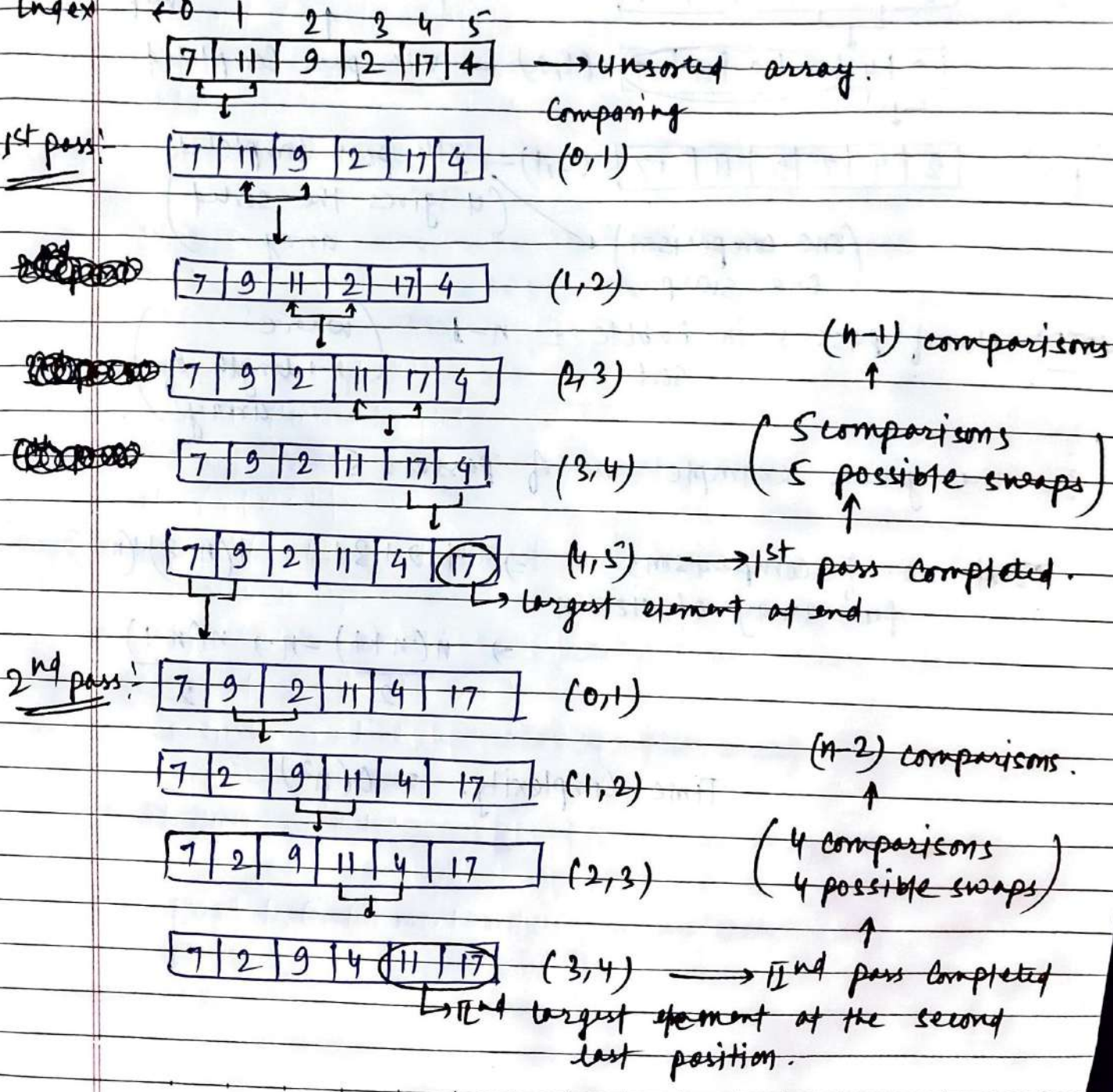
Types of Sorting :-

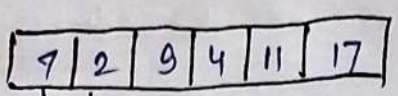
(i) Bubble sort
(ii) Selection sort
(iii) Insertion sort
(iv) Merge sort
(v) Quick sort

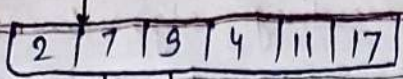
Bubble sort

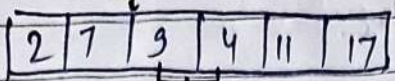
* Bubble sort is a simple sorting algorithm in which adjacent elements are compared and swapped if they are in wrong order.

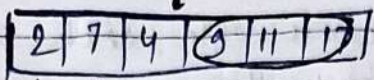
* It is called bubble sort because after each pass, the largest element bubbles up to the end of array.

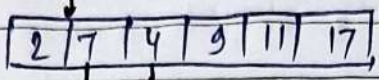


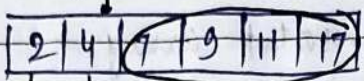
3rd pass:- 

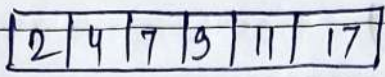

(0,1)


(1,2) (3 comparison & possible swaps)


(2,3) → IIIrd pass completed.

4th pass:- 
(0,1) (2 comparisons, 2 possible swaps)


(1,2) → IVth pass completed

5th pass:- 
(0,1) → Vth pass completed (4 gives the sorted array)

(one comparison one swap)

NOTE:- No. of passes in bubble sort = n-1 (where n = length of array)

In the given example no. of passes = 5

Total no. of comparisons for array of size n $\Rightarrow 1+2+3+\cdots+(n-2)+(n-1)$

$\Rightarrow \frac{n(n+1)}{2} - n \Rightarrow \frac{n(n-1)}{2}$

Time Complexity: $= O(n^2)$

Time Complexity :-

(i) **for worst Case :-** No. of comparisons = $\frac{n(n-1)}{2}$

Time Complexity = $O(n^2)$

(ii) **for Average Case :-** for a randomly arranged array, bubble sort still performs approximately the same no. of comparisons.

Average case time complexity = $O(n^2)$.

(iii) **for Best Case :-** Sorts the array is 1st pass → Already sorted

Total comparisons = $(n-1)$ → (For checking)

Best case time Complexity = $O(n)$

Algorithm for bubble sort :-

Step 1 :- start from the 1st element.

Step 2 :- Compare two adjacent elements.

Step 3 :- If the first element is greater than second then swap them

Step 4 :- Continue until the end of the array.

Step 5 :- Repeat the process for the remaining unsorted elements.

Step 6 :- Stop when the array is completely sorted.

Pseudo Code for bubble sort:-

Page No.
Date

```
Bubblesort (A, n)

for i=0 to n-2
   for j=0 to n-i-2

   if A[j] > A[j+1]

   swap A[j] and A[j+1].
```

C program for Bubble Sort

```
#include <stdio.h>

void PrintArray (int * A, int n) {
   for (int i=0 ; i<n ; i++) {
      print ("%d ", A[i]);
   }
   print ("\n");
}

void Bubblesort (int *A, int n) {
   int temp;
   for (int i=0 ; i<n-1 ; i++) // for no. of passes.
   {
      for (int j=0 ; j<n-i-1 ; j++) // for comparison
                                    in each pass.
      {
         if (A[j] > A[j+1]){
            temp = A[j];
            A[j] = A[j+1];
            A[j+1] = temp;
         }
      }
   }
}
```

```
int main () {
  int A[] = {12, 54, 65, 7, 23, 9};
  int n = 6;

  printArray (A, n); // Printing the array before sorting
    print+("\n");
  Bubblesort (A, n); // Function to sort the array.

  PrintArray (A, n); // Printing the array after sorting.

    return 0;
}
```

Pass	i (i=0; i<n-1; i++)	j (j=0; j<n-i-1; j++)	No. of comparisons
1	0	0, 1, 2, 3, 4	5
2	1	0, 1, 2, 3	4
3	2	0, 1, 2	3
4	3	0, 1	2
5	4	0	1

Advantages:-

* Simple and easy to understand
* Easy to implement
* No extra memory required
* Stable sorting algorithm.

Insertion Sort

* Insertion sort is a sorting algorithm that builds the sorted array one element at a time.

* It works similarly to how we arrange playing cards in our hand, we take one card and insert it into its correct position among the cards that are already sorted.

Algorithm:-

Step1:- Start from the $[I]^{nd}$ element.

Step2:- Store the current element is a variable called key.

Step3:- Compare key with the elements before it.

Step4:- Shift elements greater than key one position to the right.

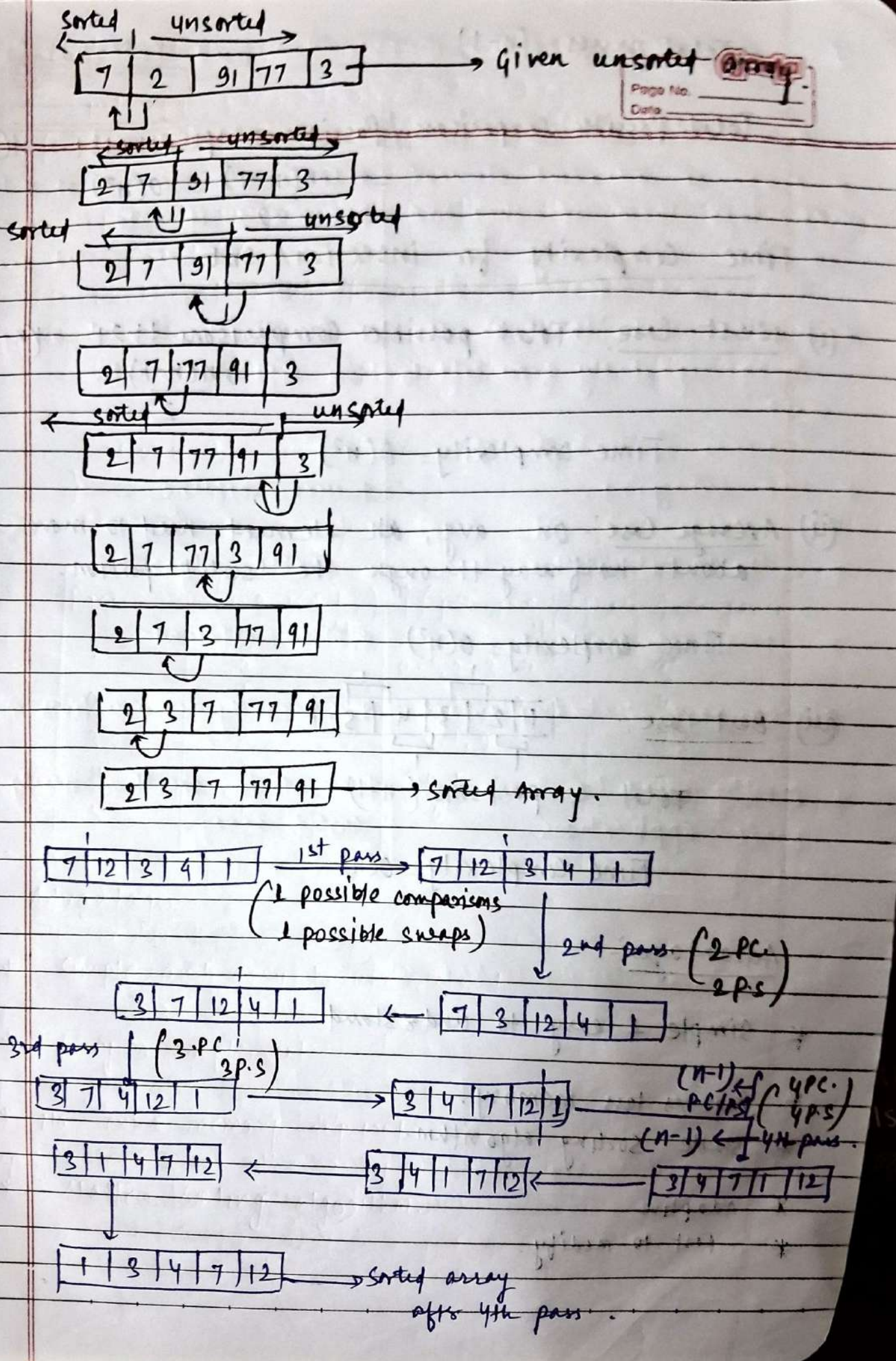
Step5:- Insert key at its correct position.

Step6:- Repeat until all elements are sorted.

Pseudo Code:-

```
InsertionSort (A, n)
    for i=1 to i=n-1
        key = A[i]
        j = i-1
        while j>0 and A[j]>key
            A[j+1] = A[j]
            j = j-1

    A[j+1] = key.
```


sorted
unsorted
7 2 91 77 3
Given unsorted array.
2 7 91 77 3
2 7 91 77 3
2 7 77 91 3
2 7 77 91 3
2 7 77 3 91
2 7 3 77 91
2 3 7 77 91
2 3 7 77 91
sorted array.
Ex:
7 12 3 4 1
1st pass
(2 possible comparisons
1 possible swaps)
7 12 3 4 1
2nd pass (2 PC. 2 PS)
7 3 12 4 1
3 7 12 4 1
3rd pass (3 PC 3 PS)
3 7 4 12 1
3 1 4 7 12
1 3 4 7 12
sorted array
after 4th pass.
3 4 7 12 1
(4 PC. 4 PS)
4th pass.
3 4 1 7 12
3 4 7 1 12

Total passes = (n-1)

Total possible comparisons / possible swaps: 1+2+3+...+(n-1)

$= \frac{n(n-1)}{2} = O(n^2)$

Time Complexity for insertion sort

(i) **Worst Case**:- Total possible comparisons = 1+2+...+(n-1)

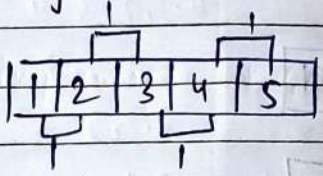
$\Rightarrow \frac{n(n-1)}{2}$

Time complexity = $O(n^2)$

(ii) **Average Case**:- On avg, element need to move about half way through the sorted portion.

Time complexity = $O(n^2)$

(iii) **Best Case**:-



Total comparisons = (n-1)

Time complexity = O(n)

Advantages:-

* Simple & easy to understand
* Requires less memory.
* Stable sorting algorithm.
* Adaptive
* Fast to modify.

C program for Insertion sort

```
#include <stdio.h>
void printArray(int *A, int n) {
  for (int i=0; i<n; i++)
  {
    printf("%d", A[i]);
  }
  printf("\n");
}
void InsertionSort(int *A, int n) {
  int key;
  int j;
for (int i=1; i<=n-1; i++) // Loop for passes.
 {
   key = A[i];
   j = i-1;

   while ((j>=0) && (A[j] > key)) { // Loop for each pass.
     A[j+1] = A[j];
     j--;
   }
   A[j+1] = key;
 }
}
int main() {
 int A[] = {12, 54, 65, 7, 23, 9};
  int n=6;
 PrintArray(A, n);
 InsertionSort(A, n);
 PrintArray(A, n);
  return 0;
}
```

1 Difference between Bubble sort and Insertion sort

Bubble sort	Insertion Sort
* Compares adjacent element and swaps them if they are in the wrong order.	* Takes one element at a time & inserts it into its correct position in the sorted part
* Usually works through the array repeatedly from left to right.	* Builds the sorted array from left to right.
* Use swapping.	* Uses shifting.
* Stable algorithm	* Stable algorithm.
* Less efficient	* More efficient
* Can perform many swaps	* Generally requires fewer moves
* Mostly used for learning/ basic applications.	* Useful for small or nearly sorted array.